PeerLens: Multi-Agent Reflective Review for Research Drafts

A MINI PROJECT REPORT FOR THE COURSE

AD23731 Foundations of Agentic AI

Submitted by

Gunavazhagan B 231501055

Jevanandham S 231501069

Nimal M G 231501110

Department of Artificial Intelligence and Machine Learning



RAJALAKSHMI ENGINEERING COLLEGE

Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

(An Autonomous Institution, Affiliated to Anna University, Chennai)

OCTOBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

BONAFIDE CERTIFICATE

Certified that this is a Bonafide Record of the mini-project work done by **Gunavazhagan B (231501055), Jevanandham S (231501069), Nimal M G (231501110)** of VII Semester, B.Tech. Artificial Intelligence and Machine Learning Department in partial fulfilment of the requirements of the course **AD23731 – Foundations of Agentic AI** during the Academic Year 2026 – 2027.

Faculty in charge

Submitted for the Practical Examination held on: .....

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ABSTRACT

Students and early-career researchers often receive detailed criticism of their work only after a supervisor, examiner, or conference reviewer evaluates the paper, when correcting flawed experiments, unsupported claims, or methodological issues can already be costly and time-consuming. This project presents **PeerLens**, an agentic AI system designed to provide authors with an on-demand, structured peer-review panel before formal submission. PeerLens is implemented as a multi-agent architecture orchestrated using LangGraph and powered by Google Gemini, comprising three specialized reviewers: a **Rigor Reviewer** that evaluates methodology, statistical reasoning, experimental design, and mathematical formulations; a **Clarity Reviewer** that identifies ambiguous terminology, structural problems, inconsistencies, and contradictions; and a **Novelty Reviewer** that examines the paper's claims against relevant published research. To improve review completeness, a page-coverage gate verifies that all pages of the manuscript have been examined, while a **Meta-Reviewer** consolidates overlapping findings, resolves reviewer-level inconsistencies, and can request additional review passes when evidence is insufficient. PeerLens follows a human-in-the-loop workflow in which the author retains control over the final assessment: each detected issue must be explicitly accepted or disputed before the final report is unlocked, disputed findings are forwarded to a **Reconsideration Agent** for further analysis, and a **Solution Generator** produces a concrete remediation plan for every accepted issue. The system's retrieval component is evaluated through three Novelty Reviewer configurations to measure the contribution of progressively stronger retrieval strategies: a **No-RAG** baseline without external retrieval, **Basic RAG**, which performs a single arXiv search followed by Gemini-embedding-based ranking using FAISS, and **Agentic RAG**, which autonomously plans search queries, identifies missing evidence, and performs additional searches for up to three rounds. Across 23 live review runs involving two research papers containing deliberately injected mistakes, PeerLens detected an average of 9.0 out of 10 mistakes in one paper and 12.0 out of 17 mistakes in the other using the default Gemini model, with every reported finding supported by a quotation from the manuscript. Evaluation with a stronger Gemini model achieved complete detection of the injected mistakes, demonstrating the potential of multi-agent orchestration, iterative retrieval, and human-controlled verification to provide practical pre-submission research feedback.

Keywords: Agentic AI, multi-agent systems, LangGraph, agentic RAG, FAISS, Gemini, human-in-the-loop, automated peer review

TABLE OF CONTENTS

BONAFIDE CERTIFICATE 2
ABSTRACT i
1. INTRODUCTION 1
1.1 Background and Motivation 1
1.2 Need for Agentic AI 1
1.3 Project Objectives 1
2. PROBLEM STATEMENT AND BUSINESS CASE 1
2.1 Problem Definition 1
2.2 Business and Real-World Context 2
3. USER AND STAKEHOLDER ANALYSIS 2
3.1 Target Users 2
3.2 Stakeholder Identification 2
3.3 User Requirements 2
4. PROPOSED AGENTIC AI SOLUTION 2
4.1 Overall Solution 2
4.2 Key Features 3
4.3 Agent-Based Approach 3
5. DATASET / KNOWLEDGE SOURCES 4
5.1 Dataset and Knowledge Source Description 4
5.2 Tools and Technologies Used 4
6. AGENT ARCHITECTURE 4
6.1 Agent Components 4
6.2 Tools 5
6.3 Memory 5
6.4 Reasoning Flow 5
6.5 Agent Interactions 5
7. AGENT WORKFLOW AND ORCHESTRATION 6
7.1 Workflow Design 6
7.2 Task Decomposition 6
7.3 Agent Coordination 7
7.4 Orchestration Implementation 7
8. AGENTIC RAG IMPLEMENTATION 7
8.1 Retrieval Architecture 7
8.2 Knowledge Retrieval 8
8.3 Generation Process 8

8.4 Retrieval-Quality Evaluation 8
9. SYSTEM IMPLEMENTATION 9
9.1 Functional Prototype Screenshots 9
9.2 Source-Code Repository 10
10. EXPERIMENTAL EVALUATION AND RESULTS 10
10.1 Evaluation Metrics 10
10.2 Testing Methodology 10
10.3 Performance Results 10
10.4 Success-Metric Achievement 12
11. COMPARISON, INNOVATION AND SCALABILITY 13
11.1 Comparison with Baseline 13
11.2 Advantages of the Proposed System 13
11.3 Innovation 13
11.4 Scalability 13
11.5 Real-World Applicability 13
12. CONCLUSION AND FUTURE WORK 14
12.1 Conclusion 14
12.2 Future Work 14
REFERENCES 15

1. INTRODUCTION

1.1 Background and Motivation

Writing a research paper involves many judgement calls: which baseline to compare against, how to split the data, how strongly to phrase a claim. Most of these are checked by someone else only at the very end, when a supervisor reads the final draft or a reviewer receives the submission, and feedback at that stage is late and expensive to act on. Peer review works because it is plural: a statistician, a domain expert and a careful reader each notice different problems. PeerLens grew out of a simple question: *could every author have a small review panel on demand, months before the real one?*

1.2 Need for Agentic AI

A single LLM prompt asking to "review this paper" gives a generic answer: one pass covers every aspect shallowly, the model cannot look anything up, and the author has no say. A useful reviewer needs **specialisation** (separate agents per quality dimension), **tool use and retrieval** (literature the agent chooses to search), **reflection** (a coordinator that notices duplicates or conflicts), **memory** (stored reviews and versions) and **a human checkpoint** (the author accepts or contests each finding). These are the capabilities that separate an agentic system from a chatbot.

1.3 Project Objectives

- Build a review panel of rigor, clarity and novelty agents with a meta-reviewer, orchestrated with LangGraph.
- Implement agentic retrieval and compare it with No-RAG and Basic RAG baselines.
- Ground every finding in a quote and page, and keep the author in the loop for every issue.
- Persist papers, reviews and versions, and evaluate the system end to end with different Gemini models.

2. PROBLEM STATEMENT AND BUSINESS CASE

2.1 Problem Definition

Given a research manuscript, the system must identify genuine methodological, presentational and novelty problems, support each finding with a quotation and page, rank findings by severity, let the author accept or dispute each one, and produce a final report with a remediation

plan per issue, without inventing problems or padding a sound paper with generic advice.

2.2 Business and Real-World Context

Colleges run mini-project, capstone and thesis reviews every semester, and supervisors spend much of that time catching the same errors: unsupported claims, missing baselines, inconsistent numbers, undefined terms. Writing assistants focus on grammar and say little about whether an experiment is valid. A first technical pass lets supervisors focus on expert questions and lets students fix routine problems early. Because PeerLens runs on a low-cost hosted model and every finding points to a quote and page, it is cheap to deploy and easy to verify.

3. USER AND STAKEHOLDER ANALYSIS

3.1 Target Users

The main users are **students** preparing mini-project, capstone or thesis reports, **researchers** preparing submissions, and **project teams** who want a stored record of findings and how each was handled.

3.2 Stakeholder Identification

Besides authors, the stakeholders are **supervisors**, who receive better-prepared drafts and a record of what the author accepted or disputed; **examiners**, who see fewer avoidable errors; the **institution**, which gains a transparent, low-cost quality check; and the **developers**, who need a modular, tested system.

3.3 User Requirements

- Upload a PDF manuscript and choose a review mode.
- Separate rigor, clarity and novelty reviews with severities, each issue quoting the paper with a page.
- Every page examined by every reviewer.
- Accept or dispute each issue; disputed issues re-examined; a downloadable report with remediation plans.
- Stored papers, reviews and versions, and visible retrieval evidence.

4. PROPOSED AGENTIC AI SOLUTION

4.1 Overall Solution

PeerLens models a conference review panel. The author uploads a draft and picks a review mode. A LangGraph workflow runs the rigor, clarity and novelty reviewers, checks that each

covered every page, and passes their findings to a meta-reviewer that merges duplicates and decides whether a reviewer should look again. The author then accepts or disputes each issue, and once all are answered the final report with remediation plans can be generated. Figure 1 shows the architecture.

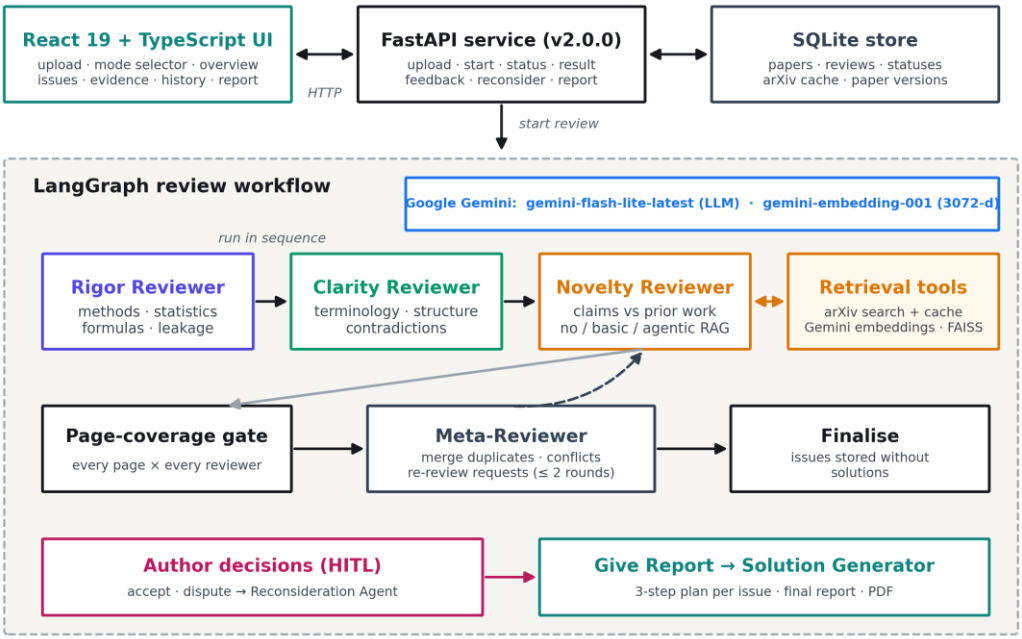


Figure 1: Overall system architecture of PeerLens

4.2 Key Features

- **Specialist panel** with a page-coverage gate and a meta-reviewer.
- **Three novelty modes** (No-RAG, Basic RAG, Agentic RAG) with every query and score visible.
- **Semantic retrieval:** Gemini embeddings ranked in a FAISS index, arXiv results cached in SQLite.
- **Human-in-the-loop:** accept, dispute and reconsider; the report unlocks only when every issue is answered.
- **Memory:** stored papers, reviews and versions, with re-review of a revised draft.

4.3 Agent-Based Approach

A multi-agent design was chosen for three reasons: each specialist can focus on one dimension with its own instructions; the agents need different capabilities (only the novelty reviewer searches outside the paper); and agents can be checked against one another, since findings from different reviewers are merged and disputed findings can be sent back. Parts that need no autonomy were kept simple: the meta-reviewer is deterministic Python, so merging and

ranking are reproducible.

5. DATASET / KNOWLEDGE SOURCES

5.1 Dataset and Knowledge Source Description

PeerLens does not train a model. Its knowledge comes from the **uploaded manuscript** (parsed page by page with pypdf), **arXiv** (live search with a SQLite cache), **Gemini embeddings** (3,072-dimensional vectors for semantic ranking) and the **Gemini model's** own background knowledge.

For evaluation we used three manuscripts. The **BrainGAN test paper** (10 pages) contains ten deliberately injected mistakes, such as an impossible accuracy of 104.2%, data leakage and a false claim of having invented DCGAN. The **Hyperspectral test paper** (18 pages) contains seventeen, which we annotated ourselves, including invented GAN and AC-GAN claims, a wrong Kappa formula and a conclusion that contradicts the results. Because the location of every mistake is known, these papers serve as ground truth. A short, carefully written **control paper** tests whether the system stays quiet when there is little to report.

5.2 Tools and Technologies Used

Table 1: Tools and technologies

Layer	Technology
Agent orchestration	LangGraph, LangChain Core
Language model	Google Gemini (gemini-flash-lite-latest by default)
Retrieval	arXiv API, Gemini embeddings, FAISS (IndexFlatIP)
Backend and storage	Python 3.11, FastAPI, Pydantic 2, SQLite, pypdf
Frontend	React 19, TypeScript, Tailwind CSS 4, Vite 8
Testing	pytest (175 tests), TypeScript compiler

6. AGENT ARCHITECTURE

6.1 Agent Components

Table 2: Agents in the PeerLens panel

Agent	Responsibility
Rigor Reviewer	Baselines, statistics, formulas, leakage, reproducibility
Clarity Reviewer	Terminology, cross-section contradictions, references, readability
Novelty Reviewer	Contribution claims against prior work, in three retrieval modes
Meta-Reviewer	Merges duplicates, detects conflicts, ranks issues, requests re-review

Agent	Responsibility
Reconsideration Agent	Judges an author's dispute: remove, reframe or uphold the issue
Solution Generator	Writes a three-step remediation plan for each issue
Revision Verifier	Checks whether earlier issues persist in a revised draft

6.2 Tools

The agents use **paper tools** (`read_paper`, `locate_evidence`, `extract_tables`), **analysis tools** for formulas, experimental claims, terminology and references, and **retrieval tools** (arXiv search with caching, embedding and FAISS ranking, evidence comparison). In normal operation the rigor and clarity reviewers send the whole paper to Gemini in one prompt; the analysis tools power a rule-based fallback, which none of our default-model evaluation runs needed.

6.3 Memory

Working memory is the LangGraph state (pages, reviewer outputs, page registry, retrieval history, re-review count). **Long-term memory** in SQLite holds papers, review records with author feedback, the arXiv cache and manuscript versions, which makes re-review of a revised draft possible.

6.4 Reasoning Flow

Each specialist receives a role prompt, the instruction to cover every page, and the paper marked page by page, and returns structured JSON with the type, page, severity, explanation, quoted evidence and recommendation for each issue. The novelty reviewer first selects the claim passages that state novelty and then, depending on the mode, searches the literature and checks whether the evidence covers what the claim requires.

6.5 Agent Interactions

The specialists interact through the meta-reviewer, which merges findings that describe the same problem on the same page and flags opposing statements as conflicts. The author interacts through decisions (Figure 2): accepting counts towards unlocking the report, while disputing sends the argument to the Reconsideration Agent.

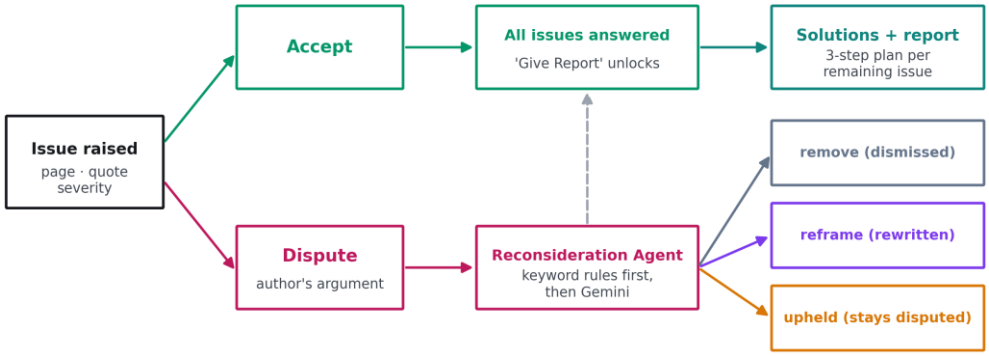


Figure 2: Human-in-the-loop decision flow for a single issue

7. AGENT WORKFLOW AND ORCHESTRATION

7.1 Workflow Design

The workflow is a LangGraph state graph (Figure 3). After the manuscript is loaded, the rigor, clarity and novelty reviewers run one after another. A coverage gate re-runs any reviewer that missed a page, the meta-review node consolidates the findings, and a conditional edge either re-runs a specialist (at most twice) or finalises the review.

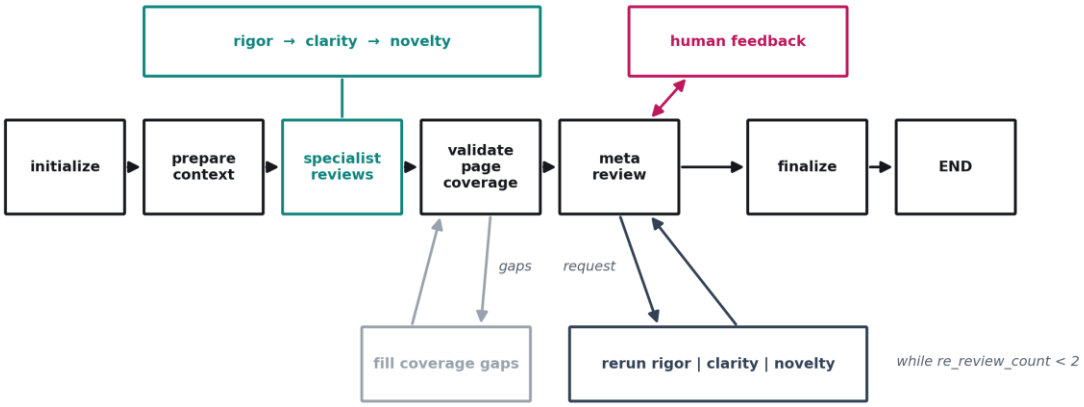


Figure 3: LangGraph workflow with the coverage gate and re-review loop

7.2 Task Decomposition

The review is split by quality dimension, each with its own prompt, tools and output schema. The novelty task is split further into claim selection, evidence requirements, query planning,

retrieval and sufficiency checking. Author feedback, reconsideration and report generation run through separate API endpoints after the graph ends.

7.3 Agent Coordination

- **Coverage:** a page registry ensures no page is skipped silently.
- **Deduplication:** similar findings on the same page are merged, keeping the higher severity.
- **Re-review:** the meta-reviewer can send an issue back to the responsible specialist.
- **Human checkpoint:** the report is generated only after every issue is answered.

7.4 Orchestration Implementation

The graph lives in `backend/graph/` (workflow, nodes, routing, state). FastAPI starts each review as a background task; the interface polls a status endpoint and then fetches the result. Fifteen REST endpoints cover upload, reviews, feedback, reconsideration, report generation and revised-paper re-review.

8. AGENTIC RAG IMPLEMENTATION

8.1 Retrieval Architecture

Retrieval serves the novelty reviewer, which supports three modes (Figure 4). **No-RAG** uses no search and only a fixed rule for sweeping "we invented" claims about well-known architectures. **Basic RAG** makes one arXiv search from an LLM-written query and ranks the results. **Agentic RAG** runs a goal-directed loop of up to three rounds.

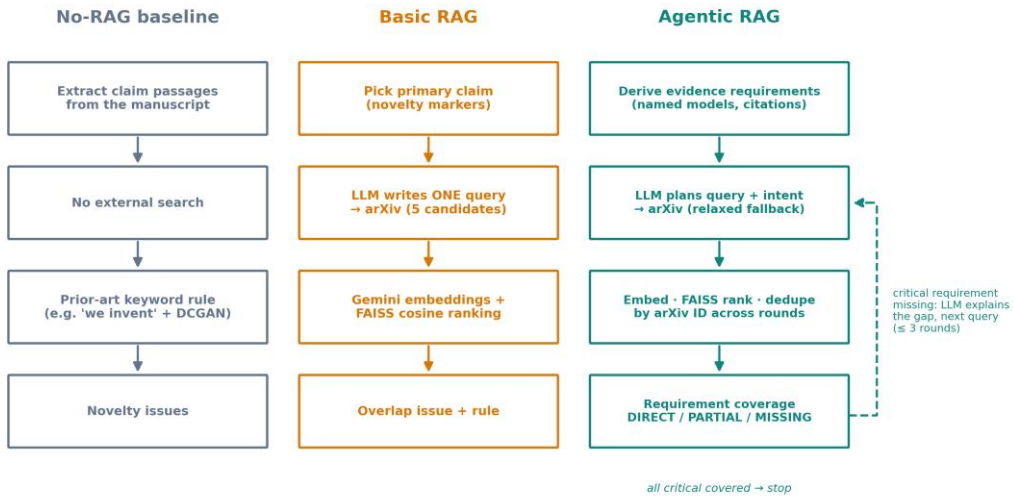


Figure 4: The three review modes of the novelty reviewer

8.2 Knowledge Retrieval

Queries are cleaned and checked against the SQLite cache; on a miss arXiv returns five candidates, and a relaxed query is tried if nothing is found. Each candidate's title and abstract is embedded with Gemini, normalised and ranked by cosine similarity in a FAISS index. In Agentic RAG, the claim is first turned into evidence requirements (each named model, cited author or arXiv ID); a requirement is covered when a retrieved paper mentions it, and the loop stops when every critical requirement is covered.

8.3 Generation Process

Gemini writes each query and its search intent from the claim, the earlier queries and the missing evidence. After the loop, the reviewer raises a literature-overlap finding that names the most similar paper and recommends how to differentiate the contribution. Every round is shown in the Evidence tab (Figure 5).

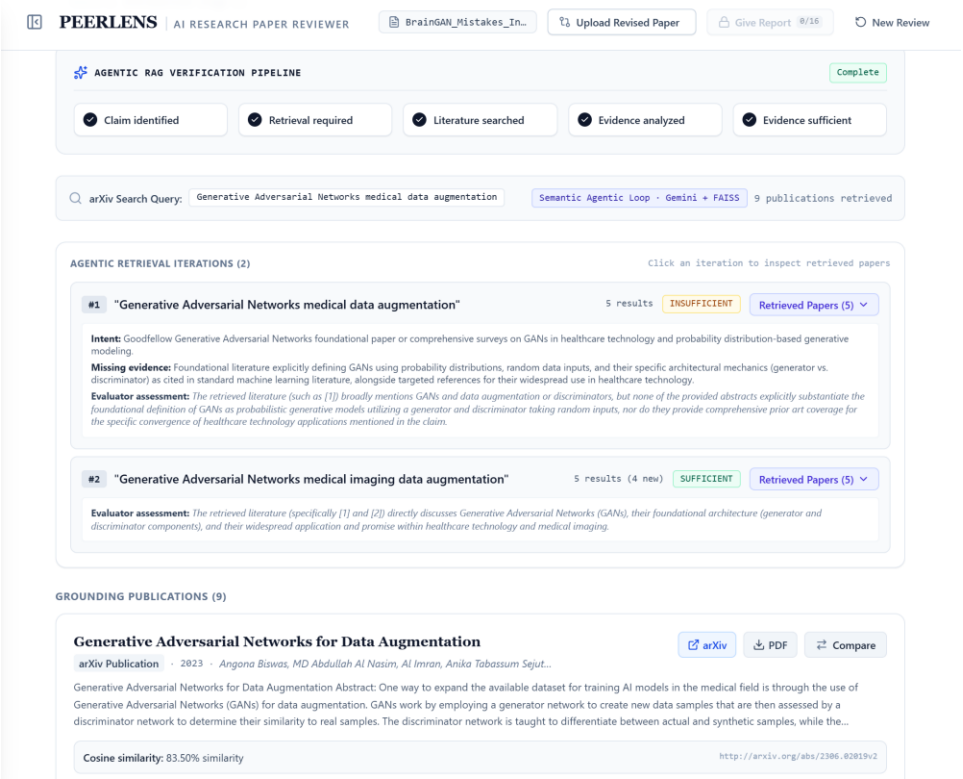


Figure 5: Evidence tab: two agentic rounds, the first insufficient and the second sufficient

8.4 Retrieval-Quality Evaluation

We re-ran V2's retrieval benchmark, which sends seven queries through the same arXiv, Gemini and FAISS pipeline and scores the top ten results against relevance labels. The first result was relevant for six of the seven queries (mean reciprocal rank 0.86) and precision at 5 was 0.69 counting partial matches. Relevant papers scored a mean cosine of 0.75 against 0.65 for irrelevant ones, a correct but narrow separation.

9. SYSTEM IMPLEMENTATION

The system is a Python backend (about 9,900 lines, plus 175 tests) and a React frontend (about 6,900 lines). The screenshots below come from live Gemini reviews of the BrainGAN paper.

9.1 Functional Prototype Screenshots

The home screen takes the manuscript and review mode, and the issue view shows the reviewer counts, overall assessment, decision progress and each issue with its quoted evidence (Figures 6 and 7).

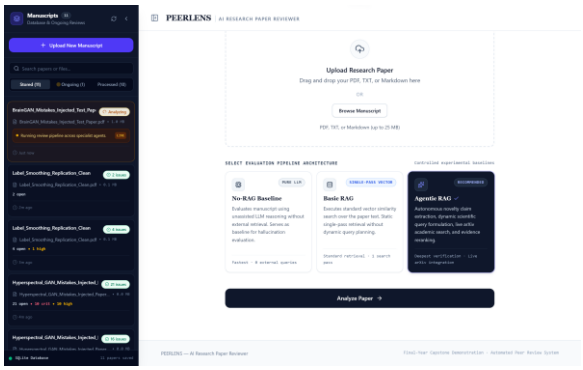


Figure 6: Home screen with review-mode selection

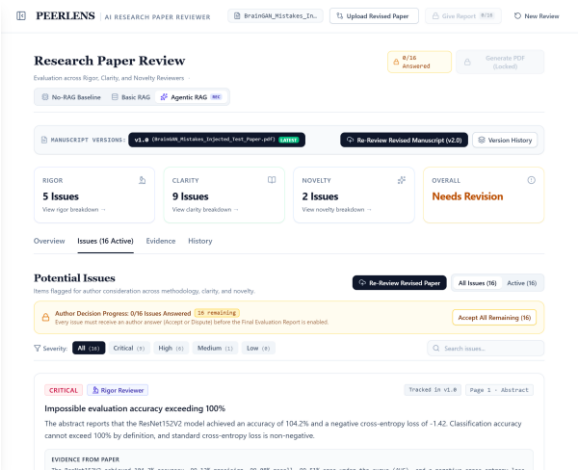


Figure 7: Issue view with decision progress

Disputing an issue opens a dialog for the author's argument, and once every issue is answered the final report lists the issues by priority with evidence and remediation (Figures 8 and 9).

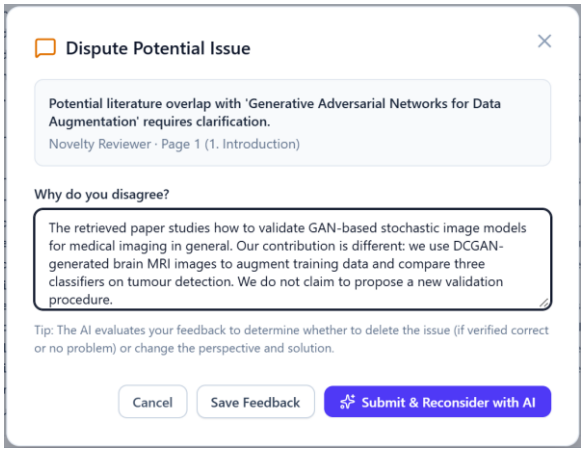


Figure 8: Dispute dialog

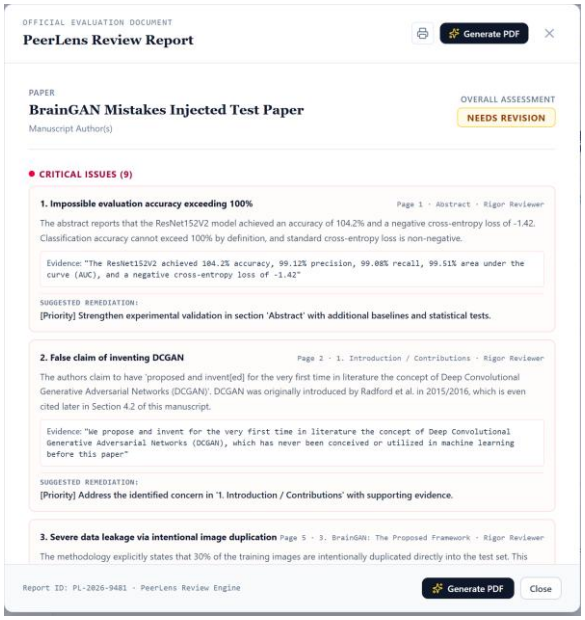


Figure 9: Generated review report

9.2 Source-Code Repository

The repository has a `backend/` folder (agents, graph, rag, tools, prompts, models, API, database, tests and the retrieval benchmark) and a `frontend/` folder with the React interface. The README documents setup and testing.

10. EXPERIMENTAL EVALUATION AND RESULTS

10.1 Evaluation Metrics

We measured **recall** of the injected mistakes (share matched by at least one issue), **issue counts** and **findings on the control paper** (false positives), **consistency** between repeated runs, **latency**, **retrieval quality**, and whether any reviewer had to use its rule-based **fallback**.

10.2 Testing Methodology

All 175 automated tests pass. For the live evaluation we ran the complete system through its API and scored each review automatically against the annotated mistakes, checking the matches by hand. We analysed 23 reviews: 17 with the default model across the three papers and three modes, and six with two other Gemini models.

10.3 Performance Results

On BrainGAN the default model found nine of ten mistakes in every run, in every mode; the

one it never found was a dataset-split arithmetic error (22 patients split into 154, 44 and 22 images). On the longer Hyperspectral paper it found between 10 and 14 of 17 (mean 12.0); the misses were again mostly arithmetic. Every reported issue carried a quote, a page and a section. Figure 10 shows which mistakes each run found.

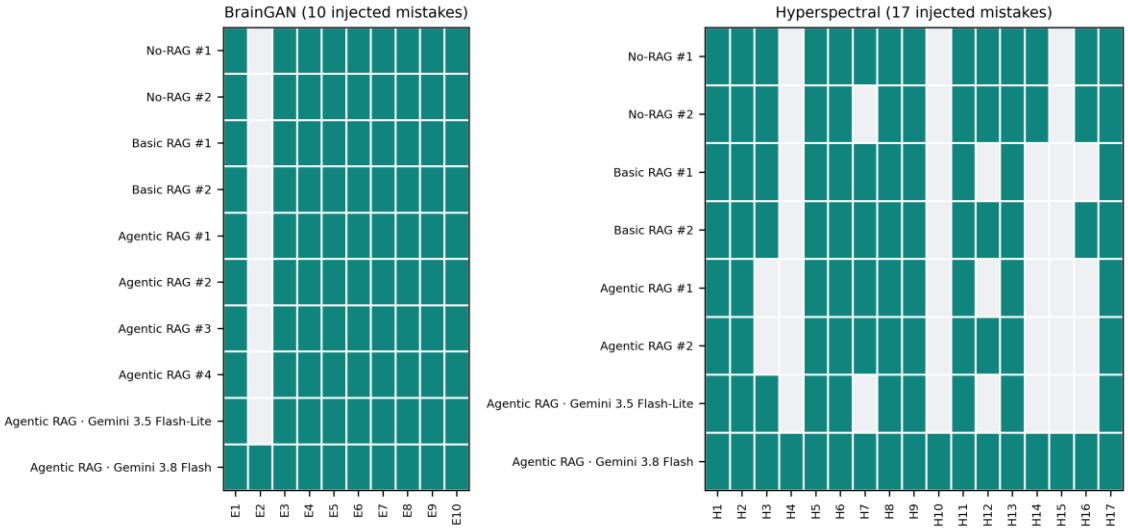


Figure 10: Detection of each injected mistake across runs

Comparison of review modes

Detection was the same in all modes on BrainGAN. On Hyperspectral No-RAG scored highest and Agentic RAG lowest, but this is run-to-run variation rather than an effect of retrieval: almost all detections come from the rigor and clarity reviewers, which receive identical input in every mode. What retrieval adds is a literature-overlap finding backed by a retrieved paper, at the cost of time (86 s against 38 s on BrainGAN).

Table 3: Results by review mode (default model)

Paper	Mode	Runs	Mean detected	Recall	Mean issues	Mean time (s)
BrainGAN	No-RAG	2	9.0/10	90%	17.0	38
BrainGAN	Basic RAG	2	9.0/10	90%	17.5	54
BrainGAN	Agentic RAG	4	9.0/10	90%	16.8	86
Hyperspectral	No-RAG	2	13.5/17	79%	20.0	40
Hyperspectral	Basic RAG	2	12.0/17	71%	19.5	42
Hyperspectral	Agentic RAG	2	10.5/17	62%	16.5	44

Comparison of Gemini models

Gemini 3.5 Flash-Lite behaved like the default model. Gemini 3.8 Flash found every injected mistake in both papers, including the arithmetic ones, but took about twice as long and raised more issues. With one run per model this is indicative, but it suggests that model choice matters more than retrieval mode for finding errors.

Table 4: Agentic RAG with three Gemini models

Model	BrainGAN (of 10)	Hyperspectral (of 17)	Control issues	Mean time (s)
Flash-Lite (default)	9.0/10 (n=4)	10.5/17 (n=2)	4	65
Gemini 3.5 Flash-Lite	9.0/10	10.0/17	5	52
Gemini 3.8 Flash	10.0/10	17.0/17	4	124

Control paper, consistency and author workflow

On the control paper the rigor and clarity reviewers raised only one or two Low or Medium remarks per run, but in both retrieval modes the novelty reviewer raised a High-severity overlap finding although the paper claims no novelty. Repeated runs agreed closely: all BrainGAN runs found the same nine mistakes, and the Hyperspectral runs had a mean Jaccard overlap of 0.81. Exercising the author workflow in the interface showed that some parts need refinement, summarised below.

Table 5: Observations from the live runs

Observation	Effect
Disputes are accepted too easily	Most arguments, even weak ones, lead to the issue being reframed instead of re-checked
Remediation plans are generic	Solutions in the report follow templates rather than being written for each issue
Overlap finding raised readily	Retrieval runs add a High-severity overlap issue, even on the control paper
Duplicate findings across reviewers	The same mistake can appear more than once when reviewers word it differently

10.4 Success-Metric Achievement

Table 6: Success metrics

Objective	Result	Status
Detect injected mistakes	BrainGAN 90% in all 8 runs; Hyperspectral 59%–82%; 100% on both with Gemini 3.8 Flash	Met
Grounded findings	Quote, page and section on every reported issue	Met
Quiet on a sound paper	Only Low/Medium presentation points, but a High novelty-overlap issue in both RAG modes	Partly
Agentic RAG vs baselines	All three work; retrieval adds evidence-backed overlap findings but does not change detection of the injected mistakes	Met
Human-in-the-loop	Works end to end; disputes of six or more words are always reframed, and solutions fall back to templates	Partly
Multiple LLMs	Three Gemini models compared on the same papers	Met
Persistence	Papers, reviews, feedback and versions stored in SQLite; accept decisions kept only in the browser	Partly
Reliability and tests	No fallback in the default-model runs; 175 of 175 tests pass	Met

11. COMPARISON, INNOVATION AND SCALABILITY

11.1 Comparison with Baseline

Compared with a single-prompt LLM review, PeerLens covers each quality dimension with a specialist, guarantees that every page is read, ties every finding to a quote and page, grounds novelty in retrieved literature, and lets the author decide on each finding, with all reviews and versions stored.

11.2 Advantages of the Proposed System

- **Verifiable output:** each issue points to a quote and a page, so it can be checked in seconds.
- **Observable retrieval:** every query, missing requirement and score is visible.
- **Low cost:** a free-tier model reviewed a ten-page paper in about a minute and found nine of ten mistakes.

11.3 Innovation

PeerLens combines **requirement-driven retrieval**, which keeps searching until each piece of evidence a claim needs is found; a **coverage contract** that makes reading the whole paper a checked property; and **the author as a gate**, so the final report records human decisions rather than raw model output.

11.4 Scalability

Reviews run as background tasks, the arXiv cache avoids repeated calls, the FAISS store can be made persistent, storage can move from SQLite to PostgreSQL for many users, and any Gemini model can be chosen through configuration.

11.5 Real-World Applicability

PeerLens can serve as a pre-submission check for student reports: a student uploads a draft before the first review meeting, works through the issues and brings the report to the supervisor. The same workflow suits research groups and editors screening manuscripts, supporting human judgement rather than replacing it.

12. CONCLUSION AND FUTURE WORK

12.1 Conclusion

We built an agentic system that gives authors a review panel before the real one. Specialist reviewers, a page-coverage gate and a meta-reviewer read the whole paper and found nine of ten injected mistakes in every BrainGAN run and on average 12.0 of seventeen in a longer paper, each with a quote and page, using a free model in about a minute; a stronger Gemini model found every mistake. The agentic retrieval loop works as designed and its reasoning is visible to the author. The evaluation also showed where the system can improve, especially in how disputes and remediation plans are handled.

12.2 Future Work

PeerLens is a working first version, and we see several clear directions in which to grow it:

- **More reviewer agents:** add specialists for ethics and reproducibility, statistics, writing style and citation formatting, so that the panel covers more of what a real review committee checks.
- **Wider literature search:** search Semantic Scholar, IEEE Xplore and Google Scholar alongside arXiv, and follow citation links, so that novelty checks reach journal and conference papers too.
- **Supervisor and team workspace:** user accounts, shared reviews and a supervisor view where a guide can see a student's draft, the issues raised and how each one was handled.
- **Smarter revision tracking:** a side-by-side view of each draft that highlights what changed and shows, issue by issue, how the paper improved across versions.
- **More document types and languages:** support Word and LaTeX files, theses, project reports and grant proposals, and reviews in regional languages.
- **Integration with college workflows:** connect PeerLens to the department's project submission process and learning platforms, so a draft can be checked before it reaches the examiner.
- **Larger evaluation:** test PeerLens on many more papers and compare its findings with real supervisor and reviewer feedback, to measure and improve its usefulness.

REFERENCES

[1] LangChain. LangGraph documentation. https://langchain-ai.github.io/langgraph/

[2] Lewis, P. et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. NeurIPS 33.

[3] Johnson, J., Douze, M. and Jégou, H. (2019). Billion-scale similarity search with GPUs. IEEE Transactions on Big Data.

[4] Google. Gemini API documentation. https://ai.google.dev/gemini-api/docs

[5] Yao, S. et al. (2023). ReAct: synergizing reasoning and acting in language models. ICLR.

[6] Liang, W. et al. (2024). Can large language models provide useful feedback on research papers? NEJM AI.

[7] arXiv API User Manual. https://info.arxiv.org/help/api/